LogiEdge: Edge AI for Cold-Chain Monitoring on FreightBridge Trucks

AIML ZG535 Machine Learning on Edge (EC-I)

Group 15

Student Details:

BITS ID	Name	Contributions
2025ae05381	Disha Shetty H	Equal (25%)
2025ae05382	Murali Krishna Sai Garisapati	Equal (25%)
2025ae05420	Rajan Yadav	Equal (25%)
2025ae05680	Santosh Kumar Behera	Equal (25%)

Repository: https://github.com/santosh-2025ae05680/logibridge

Introduction

FreightBridge operates 85 refrigerated trucks carrying vaccines and medicines, and has already lost ₹28 lakh of vaccine to a refrigeration failure that went undetected. This report describes LogiEdge, an edge pipeline that classifies the cargo compartment as Normal, Warning or Critical on the truck itself, from temperature, compressor vibration and door events. It alerts without depending on continuous cellular coverage and can be updated and monitored across the fleet. All sensor data is simulated, so the results show that the pipeline works end to end, not how accurate it would be on real trucks. Sections 1 to 5 cover the deployment context and hardware, the sensor pipeline and MQTT design, model deployment, optimisation, and MLOps monitoring. Every figure quoted comes from the project's own scripts and from the 4 November terminal log (evidence/latest_run_2026-11-04.txt) in the repository.

1. FreightBridge Deployment Context

Latency. A failed refrigeration unit can raise cargo temperature by about 1 °C per minute, and the brief requires an alert within 90 seconds of a fault signature. Indian 4G round-trip latency averages 62–79 ms by operator (Opensignal [1]), so cloud inference would be fast enough while connected; the problem is the tail, where handovers, retries and loss of coverage make the delay unbounded. LogiEdge therefore decides on the truck. In our 4 November run the first Critical window closed 18 seconds after fault onset.

Bandwidth. Raw three-axis vibration at 500 Hz with 4-byte values is 6,000 bytes per second, or 518.4 MB per day; 1 Hz temperature adds 0.35 MB and door events about 0.002 MB, giving 518.75 MB per truck per day (₹51.87, or ₹1.32 lakh per month for 85 trucks). With edge inference only alerts leave the truck: ten 300-byte alert records are about 0.003 MB per day, roughly 173,000 times less.

Connectivity. The Nashik–Aurangabad route loses coverage for 35–90 minutes at seven locations. A cloud-only system is blind for the whole gap, and at 1 °C per minute even 35 minutes can spoil vaccines unnoticed. Our broker, preprocessing, inference and alert log keep running offline, and a sync service delivers queued alerts when the uplink returns; two alerts were delivered after reconnection in our run.

Privacy. Because inference runs on the truck, raw temperature and vibration streams never leave the vehicle. FreightBridge can therefore commit contractually to data minimisation: the only cargo data held off the truck is the alert log, sent over TLS to an access-controlled, audited store that also serves as the temperature chain-of-custody record.

Hardware and the Constraint Triangle. We placed the three options on the performance–power–cost triangle (Table 1) using vendor specifications [2]–[4]. Power is the dominant vertex: the 10 W AI budget from the 12 V supply is the only pass/fail limit, while every option meets the 90-second requirement by a wide margin. The Jetson Orin Nano Super breaks the power budget by 50% and costs ₹38.25 lakh for the pilot and ₹119.25 lakh at 265 trucks, for 67 TOPS this model cannot use. The STM32H7 MCU wins on power (0.4 W) and cost (₹2.98 lakh pilot) but cannot run the Linux, Docker, Mosquitto and Ansible stack that our offline broker, container OTA and MLOps tasks depend on. We select the Raspberry Pi 5 with AI HAT+: 7.5 W leaves 2.5 W of headroom, it meets the 90-second requirement, and it costs ₹12.75 lakh for the pilot and ₹39.75 lakh at full scale, one third of the Jetson.

Table 1: Hardware options

Option	Hardware	Power	Price per truck	Within 10 W	Pilot (85)	Full (265)
1	Raspberry Pi 5 + AI HAT+ (Hailo-8L)	7.5 W	₹15,000	Yes	₹12.75 lakh	₹39.75 lakh
2	Jetson Orin Nano Super	15 W	₹45,000	No	₹38.25 lakh	₹119.25 lakh
3	STM32H7 custom MCU	0.4 W	₹3,500	Yes	₹2.98 lakh	₹9.28 lakh

Figure 1. Architecture of the LogiEdge proof-of-concept.

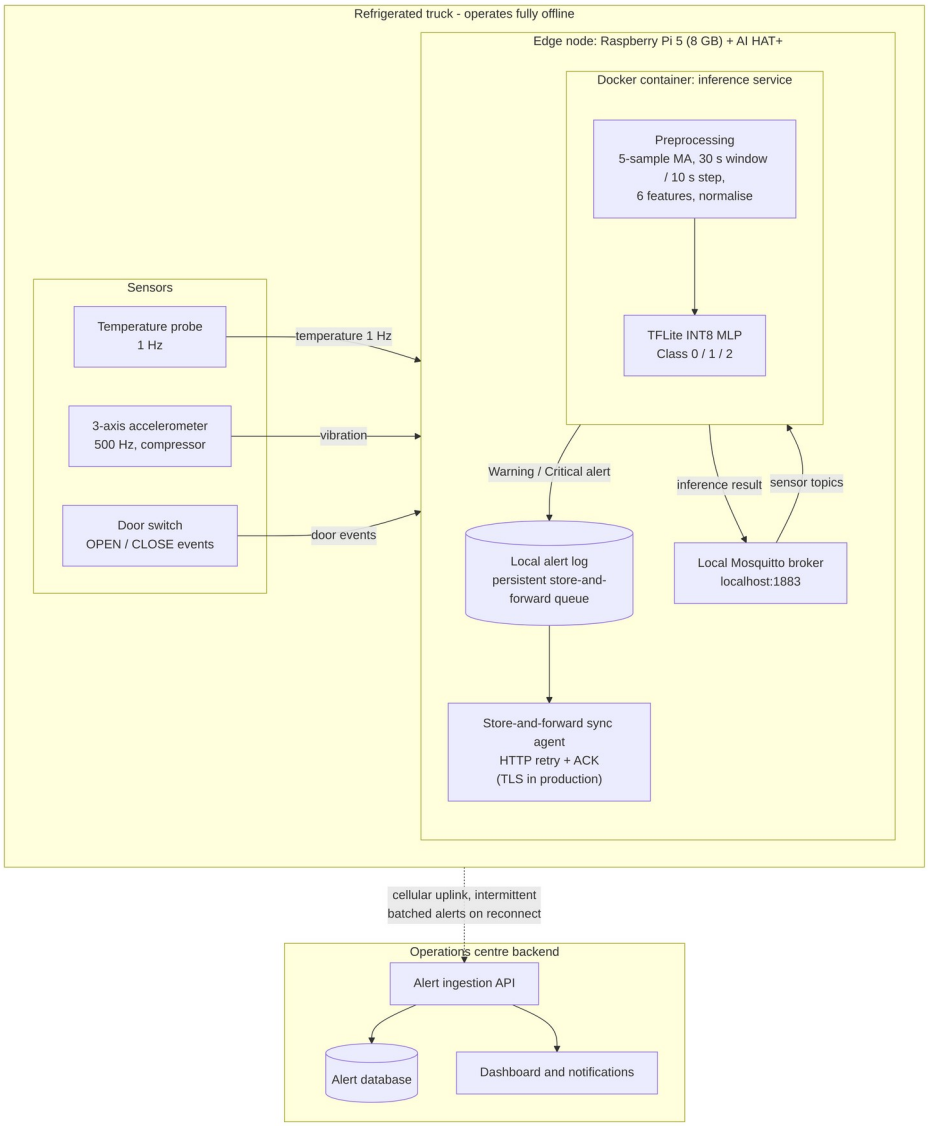


Figure 1 shows the edge node. Sensors publish to a local Mosquitto broker; the containerised inference service preprocesses, classifies and writes Warning and Critical alerts to a persistent local log; a store-and-forward agent sends them to the operations-centre backend when the cellular uplink returns.

Roofline [8]. With 45 MFLOP and 18 MB per inference, arithmetic intensity is 2.50 FLOP/byte. The Pi 5 CPU ridge point is 16 GFLOP/s ÷ 12 GB/s = 1.33 FLOP/byte, so the workload is compute-bound (2.81 ms compute against 1.50 ms memory time). Latency therefore improves by reducing arithmetic work, through INT8 quantisation and structured pruning (our variants M2 and M3), not by reducing memory traffic alone.

2. Sensor Pipeline and MQTT Design

Simulator. We used a Python sensor simulator to publish temperature at 1 Hz around the 4 °C setpoint, vibration RMS at 0.5 Hz, and discrete OPEN/CLOSE events. The available modes are none, temp_drift, vibration and combined. The temperature mode adds a linear drift of 0.08 °C per reading, while the vibration fault changes the RMS distribution toward 1.2 g. A fixed seed makes the generated sequence repeatable, which helped us compare results and investigate failures without changing the sensor inputs

between tests. In the final live run, we kept the simulator at normal speed (1×) rather than accelerating time. The combined anomaly ran from second 120 to 240; the full 1,260-second sequence published 1,260 temperature values, 630 vibration values and four door messages. The simulated RMS feed is not the raw 500 Hz signal used in the separate bandwidth exercise.

Our MQTT topics use logibridge/trucks/{truck_id}/ as the common root. Temperature and vibration RMS are published with QoS 0 [5], since the next reading quickly follows. Door, status and alert messages use QoS 1 to avoid losing important state changes; inference outputs use QoS 0 because each 10-second result supersedes the previous one. We keep status retained for late subscribers. Within the test, Mosquitto is isolated on the logibridge-net Docker network; the HTTP alert synchroniser is separate from MQTT and uses local JSONL plus an acknowledgement journal.

Figure 2. MQTT topic tree.

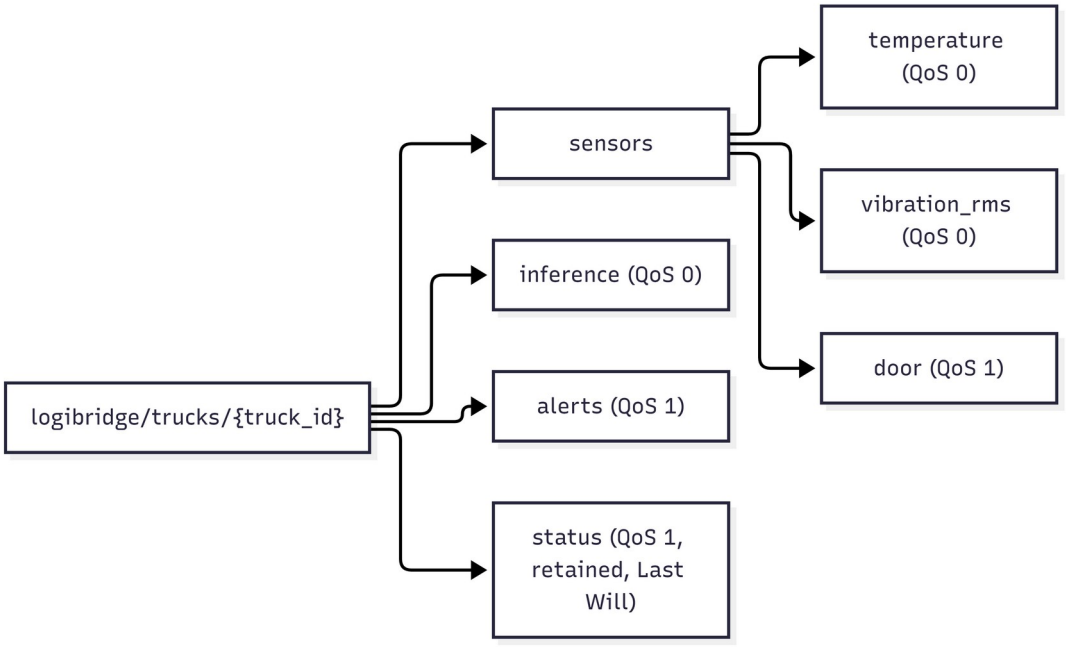


Table 2: QoS per topic

Topic	Rate	QoS	Retained
sensors/temperature	1 Hz	0	no
sensors/vibration_rms	0.5 Hz	0	no
sensors/door	on event	1	no
inference	every 10 s	0	no
alerts	on Warning or Critical	1	no
status/{client}	on change	1	yes (Last Will)

Our temperature and vibration streams first pass through separate five-sample moving averages. We then form 30-second sliding windows at 10-second steps, rather than waiting for disjoint blocks; the overlap makes it easier to observe an event beginning near a window boundary. Each window supplies six features: temperature mean, standard deviation and rate of change in °C per minute, plus vibration RMS, peak and kurtosis. We apply the saved training mean and standard deviation to each feature and clip extreme normalised values to ±10. Runtime processing loads training_stats.npy rather than

recalculating statistics from abnormal live traffic. This matters because adapting the baseline during a failure could make the very fault we need to detect appear normal.

We ran the required shifted-normalisation experiment on the 57 validation windows (Table 3). With the ±10 clip used at runtime, FP32 and INT8 accuracy stayed at 100% under +3σ and −3σ. The reason is the class margin: Warning and Critical windows lie more than 15σ from the Normal mean on the temperature features, so they are clipped at +10 and a 3σ shift cannot move them across the decision boundary. Without clipping, FP32 fell to 40.35% even with correct statistics, because z-scores in the hundreds lie far outside its training range; INT8 was unaffected because its input quantisation saturates like a clip. The clip is therefore a required part of preprocessing. Evidence: training/results/stats_shift_results.csv.

Table 3: Validation accuracy (57 windows) under shifted statistics

Model	Clip	Correct statistics	+3σ	-3σ
FP32 Keras	±10	100.0%	100.0%	100.0%
FP32 Keras	off	40.4%	40.4%	40.4%
INT8 TFLite	±10	100.0%	100.0%	100.0%
INT8 TFLite	off	100.0%	100.0%	100.0%

Data fusion. We chose feature-level fusion: the temperature and vibration processing steps each produce their own measurements, and we concatenate those measurements into one six-value input before classification. This preserves information from both sensors while avoiding the work of synchronising every raw sample across streams with different rates. Data-level fusion would require resampling or padding the high-rate waveform and lower-rate temperature values to a common timeline. That adds traffic and memory without a clear benefit for our small MLP. Decision-level fusion would require separate classifiers and a rule to combine their outputs, increasing deployment and calibration work. Door events remain a separate operational signal in the demonstration rather than being silently counted as a seventh model feature. This separation keeps the six-feature architecture reproducible and makes it clear which evidence actually drives the three-class prediction.

3. Model Deployment and Pipeline Mapping

Table 4. Our ten-stage Edge ML pipeline, mapped to the project.

#	Stage	What happens in LogiEdge
1	Problem definition and constraints	Three classes, a 90 s alert SLA, a 10 W power budget and fully offline operation set the requirements.
2	Data acquisition	A simulator publishes temperature, vibration RMS and door events to the local Mosquitto broker with per-topic QoS.
3	Preprocessing	A 5-sample moving average, 30 s windows with a 10 s step, and z-scoring with frozen training_stats.npy and a clip at ±10.
4	Feature engineering and fusion	Six features (temperature mean, standard deviation, rate; vibration RMS, peak, kurtosis) fused at feature level.
5	Model training	A 6-32-16-3 MLP (803 parameters) trained on 285 windows from three simulator modes, with a time-ordered split and a two-window purge.
6	Evaluation and validation	100% accuracy on 57 held-out windows (gate 88%) and 100% Class 2 recall (gate 95%), plus the 3σ shifted-statistics experiment.
7	Optimisation	INT8 post-training quantisation (M2, 3.38 KB) and 35% neuron pruning with INT8 (M3, 2.94 KB), compared by benchmark and Pareto analysis.
8	Conversion and packaging	TFLite models inside a python:3.11-slim container with the model as the last layer and a MODEL_PATH override.
9	Deployment and OTA update	A local registry with versioned tags and a seven-task Ansible playbook; a canary rollout is recommended.
10	Monitoring and maintenance	PSI on P(Normal) over the last 100 inferences every 60 s, alerting above 0.25, with a reference tied to the deployed model.

The ten pipeline stages are mapped to LogiEdge in Table 4. The dataset was produced by running the simulator model for 20 minutes in none mode, 15 minutes in temp_drift and 15 minutes in combined, through the same preprocessing code as the live service, and split 80/20 in time order with a two-window purge so that overlapping windows cannot leak into validation. This gave 285 labelled windows, 228 for training and 57 held out (23 Normal, 17 Warning, 17 Critical). We trained a 6–32–16–3 multilayer perceptron with 803 parameters and exported three deployment candidates. Each model classified all 57 held-out windows correctly, including all 17 Critical windows, which shows that conversion preserved the decisions on this dataset.

The inference service (preprocessing, TFLite inference, then publishing to logibridge/trucks/{truck_id}/inference) is packaged on python:3.11-slim. The Dockerfile installs all pip dependencies first, then copies the code and training_stats.npy, and copies model.tflite as the last layer; the MODEL_PATH environment variable selects a model variant without a rebuild, which is how Ansible points the container at /opt/logibridge/model.tflite. Our M2 baseline uses image tag 1.0; we rebuilt image 1.1 with the M3 model and compared the registry layers. The base image and the 72 MB dependency layer were reused; only three small application layers, 5.73 KB in total including the model, were new. The compressed image is 119.649 MB per truck, so for 85 trucks a full pull is 10,170.18 MB against 0.4875 MB with cache reuse, a 99.9952% saving, assuming the older layers remain cached on every vehicle.

Table 5: Bandwidth for 85 trucks

Update method	Per truck	85 trucks	Cost at ₹0.10/MB
Full image pull	119.649 MB	10,170.18 MB	₹1,017.02
Layer-cache OTA	0.00573 MB	0.4875 MB	₹0.0487

For the brief's OTA scenario (280 KB INT8 model every six weeks, 85 trucks, ₹0.10/MB), full replacement costs ₹2.380 per cycle (23.80 MB), a ten-truck canary followed by the other 75 costs ₹2.392 including health telemetry, and on-device shadow mode costs ₹2.440. The difference is under one rupee per year, negligible against a single ₹28 lakh spoilage event, so safety decides. We recommend canary. Full replacement exposes all 85 trucks to an unproven model at once, and with 35–90-minute coverage gaps a faulty model could miss breaches fleet-wide before anyone notices. Shadow mode exposes no trucks, but Critical events are rare in a week of normal running, so it cannot measure Critical recall, and uploading every shadow output over rural M2M links costs ₹53.79 per cycle, 22 times the canary. A canary limits exposure to ten trucks, which keep the previous model for local rollback without a fresh download. The canary trucks would report alert rates and PSI for three days before the remaining 75 are updated.

Table 6: Cost per update cycle (280 KB model)

Strategy	MB per cycle	₹ per cycle	₹ per year	Trucks exposed
Full replacement	23.80	2.380	20.7	85
Canary (10, then 75)	23.92	2.392	20.8	10
Shadow, on-device comparison	24.39	2.440	21.2	0
Shadow, upload all outputs	537.88	53.788	467.4	0

4. Optimisation and Pareto Analysis

We compared three TFLite variants of the same MLP. M1 is the FP32 baseline. M2 is full-INT8 post-training quantisation with TFLiteConverter (DEFAULT optimisation, INT8 input and output), calibrated on 228 training windows, above the 200-sample minimum. M3 applies 35% structured pruning with tensorflow_model_optimization: a PolynomialDecay schedule [6] raised sparsity from 0 to 35% over 24 of 30 fine-tuning epochs on whole-neuron blocks. We then removed the pruned neurons, giving a 6–21–10–3 network with 400 instead of 803 parameters, and applied the same INT8 quantisation.

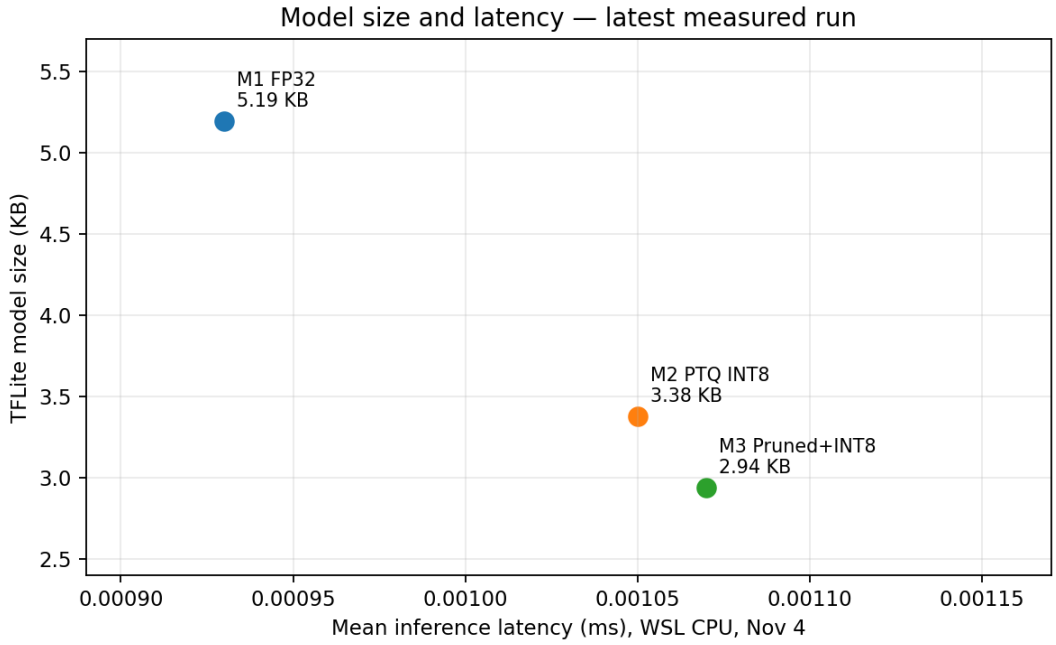
Each variant ran five interleaved trials of ten excluded warm-up calls and 200 timed calls on our 16-logical-core laptop under WSL. Energy is E = P × t, with P = 28 W TDP × one busy core out of 16 = 1.75 W; psutil reported 100% of one core during timing. Accuracy and Critical recall were measured on the same 57 held-out windows for all three variants.

Table 7: Benchmark results

Variant	Mean latency (ms)	p95 latency (ms)	File size (KB)	Accuracy (%)	Energy (mJ)
M1 FP32	0.00093	0.00109	5.19	100.00	0.001626
M2 PTQ INT8	0.00105	0.00112	3.38	100.00	0.001835
M3 Pruned+INT8	0.00107	0.00108	2.94	100.00	0.001876

M1 had the lowest mean latency (0.00093 ms), and M3 the smallest file (2.94 KB) and lowest p95 latency (0.00108 ms). The Pareto analysis (Figure 3) keeps M1 and M3 as non-dominated, with M2 dominated by M3. The latency differences are below 0.2 µs and within the trial-to-trial spread, so on a real device storage and memory decide between the variants. Relative to M1, the INT8 variants cut working tensor memory by 71% (M2) and 84% (M3).

Figure 3. Latest WSL CPU model-size and latency comparison (4 November 2026).



Observed trade-off: M1 is fastest; M3 is smallest. Both appear on the size/mean-latency frontier.

Recommendation: deploy M3. Latency budget: the 90-second SLA must cover the 30-second feature window, up to 10 seconds until the next step, and alert logging and publication (we allow 5 seconds), leaving about 45 seconds per inference; the service must also keep pace with one inference every 10 seconds. M3's measured invocation inside the live service was 0.1–0.2 ms, more than 200,000 times inside that budget, and in our run the first Critical window closed 18 seconds after fault onset. Memory: M3 needs 2.94 KB of storage and a 545-byte tensor arena (M1: 5.19 KB and 3,452 bytes), negligible against the Pi 5's 8 GB LPDDR4X RAM and microSD storage; it would even fit an STM32H7's 2 MB Flash and 1 MB SRAM, so the model does not constrain the hardware choice. Class 2 recall: M3 classified all 17 held-out Critical windows correctly (100%, above the 95% requirement), with 100% overall accuracy. M3 therefore meets all three conditions with the smallest footprint, and its smaller file also shortens every OTA transfer. We would confirm latency and recall on a physical Pi 5 during the canary phase.

5. MLOps Monitoring and Reflection

We built the M3 reference from 300 clean Normal windows, binning the Normal-class probability (p_normal) into the four required bins for the Population Stability Index [7]. We monitor p_normal rather than the maximum softmax value, because a confident Critical prediction also has a high maximum, so maximum confidence barely moves when the model switches from Normal to Critical. In the 4 November run (1× speed, 1,260 seconds, combined fault from second 120 to 240), PSI over the latest 100 predictions was 0.000 before the fault, crossed 0.25 at the next 60-second check (2.369), peaked at 4.604 and returned to 0.000 once the fault windows had left the buffer (Table 8).

Table 8. Actual PSI transition during our real-time run on 4 November 2026.

Phase	Time	PSI	Status
Baseline	16:01:50	0.000	ok; n=8
First alert	16:02:50	2.369	ALERT; n=14
Peak	16:03:50	4.604	ALERT; n=20
Declining	16:14:50	0.932	ALERT; n=86
Near recovery	16:19:50	0.313	ALERT; n=100
Recovered	16:20:50	0.000	ok; n=100
Delivered alerts	after receiver restart	2	ACK and stored

Drift type. The injected fault is sudden (abrupt) covariate drift: the input distribution of the temperature and vibration features jumped, while the relationship between features and class was unchanged, so it is not concept drift [9]. PSI observes it as prediction drift in the output distribution, and the drift reversed fully when clean data returned.

Our seven-task Ansible playbook deployed M3 and its reference: the first run reported ok=7 changed=3, and an unchanged second run reported ok=6 changed=0 skipped=1 (the stop task runs only when the model or reference changes), confirming idempotency. Because task four is conditional, the second run neither stopped nor restarted the service, and the container kept processing MQTT data.

Technical difficulty. Showing recovery in real time was harder than showing drift. With one inference every 10 seconds, a 100-prediction buffer takes about 1,000 seconds to flush, and PSI only falls below 0.10 when at most one abnormal prediction remains. Our first 320-second run detected drift but ended before recovery, so we moved to a 1,260-second real-time schedule with a two-minute fault. Recovery time is set by buffer length and inference interval, not by when the fault ends.

Architectural change. We would add a deterministic temperature-threshold rule alongside the model. Testing the deployed models after the run showed that they respond to temperature rising rather than to its level: a vibration-only fault and a temperature held 5 °C above setpoint were classified Normal in most windows, because the training recipe contains only steadily rising drift. A rule on the filtered temperature (1–3 °C off setpoint gives Warning, more than 3 °C gives Critical) would enforce the class definitions directly, while the MLP continues to detect compound faults early.

Conclusion

Our group demonstrated MQTT sensor streaming, 30-second feature windows, TFLite classification, locally stored alerts, delivery after operations-centre reconnection, an OTA update that reused every dependency layer, repeatable Ansible deployment and PSI drift monitoring. In the 4 November run the first Critical window closed 18 seconds after fault onset, PSI rose from 0.000 to 4.604 and returned to 0.000, and two Critical alerts queued during the outage were delivered after reconnection. M3 is the smallest model and meets the Critical-recall requirement. Before a real fleet deployment we would add the temperature-threshold rule, measure latency and power on a physical Pi 5, validate on recorded truck data, and test across genuine cellular outages.

References

[1] Opensignal (2019). India Mobile Network Experience Report, April 2019. Opensignal Ltd.

[2] Raspberry Pi Ltd. Raspberry Pi 5 product brief and Raspberry Pi AI HAT+ product brief. raspberrypi.com.

[3] NVIDIA Corporation. Jetson Orin Nano Super Developer Kit. https://marketplace.nvidia.com/en-us/enterprise/robotics-edge/jetson-orin-nano-super-developer-kit

[4] STMicroelectronics. STM32H742/743/753 datasheet (Arm Cortex-M7, up to 2 MB Flash, 1 MB RAM). st.com.

[5] OASIS (2014). MQTT Version 3.1.1, OASIS Standard. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[6] Google. TensorFlow Lite post-training quantization guide and TensorFlow Model Optimization Toolkit pruning guide. tensorflow.org.

[7] Siddiqi, N. (2006). Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring. Wiley.

[8] Williams, S., Waterman, A. and Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65–76.

[9] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. and Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys, 46(4), Article 44.